```
PS C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests>  py -3.13
test_autograd_integration.py

================================================================================
🚀 ЗАПУСК ИНТЕГРАЦИОННОГО ТЕСТА: ПЕРЕДАЧА ГРАДИЕНТОВ ИЗ
LOCALRUNNER В LINEAR
================================================================================
Тестирование на устройстве: NVIDIA GeForce RTX 2060

……
🔍 ТРАССИРОВКА ТЕКСТОВОГО JIT-БЛОКА (RUNNER LOG):
……
Обнаруженные объекты в области видимости forward():
  • [Tensor] x              -> size: 32, shape: [1, 4, 8]
  • [Tensor] memory_state   -> size: 32, shape: [4, 8]
  • [Tensor] proj_vectors_3d -> size: 32, shape: [1, 4, 8]
  • [Tensor] gate_vectors   -> size: 32, shape: [4, 8]
  • [Tensor] output         -> size: 32, shape: [4, 8]

Компиляция строк выражений в Си-код:
  • gate_A       = nnclloc.sigmoid(gate_vectors)
  • memory_state = memory_state * gate_A
  • output       = memory_state + gate_A
……

4 errors generated.
Traceback (most recent call last):
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_autograd_integration.py",
line 147, in <module>
    test_skvoznoi_autograd_pipeline()
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_autograd_integration.py",
line 95, in test_skvoznoi_autograd_pipeline
    output_cl = model(x_cl, memory_cl)
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\core\module.py", line 77, in
__call__
    return self.forward(*args, **kwargs)
           ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_autograd_integration.py",
line 57, in forward
    self.runner(f"""
    ~~~~~~~~~~~~^^^^^
        gate_A = nnclloc.sigmoid(gate_vectors)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        memory_state = memory_state * gate_A
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        output = memory_state + gate_A
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    """)
    ^^^^
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\compiler\nncl_compiler.py", line
243, in __call__
    program = JITProgramCache.get_or_compile(self.ctx, full_forward_code)
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\compiler\nncl_compiler.py", line
17, in get_or_compile
    cls._cache[kernel_code] = cl.Program(ctx, kernel_code).build()
                              ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^
```

```
  File "C:\Users\Дмитрий\AppData\Local\Programs\Python\Python313\Lib\site-packages\
pyopencl\__init__.py", line 580, in build
    self._build_and_catch_errors(
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^
        lambda: self._prg.build(options_bytes, devices),
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        options_bytes=options_bytes)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Дмитрий\AppData\Local\Programs\Python\Python313\Lib\site-packages\
pyopencl\__init__.py", line 638, in _build_and_catch_errors
    raise err
pyopencl._cl.RuntimeError: clBuildProgram failed: BUILD_PROGRAM_FAILURE -
clBuildProgram failed: BUILD_PROGRAM_FAILURE - clBuildProgram failed:
BUILD_PROGRAM_FAILURE

Build on <pyopencl.Device 'NVIDIA GeForce RTX 2060' on 'NVIDIA CUDA' at
0x20c395d40a0>:

<kernel>:175:404: error: redefinition of parameter 'gate_A'
        __kernel void text_forward_7829587341497687853(__global float* gate_vectors, const int
gate_vectorsSize, __global float* grad_gate_vectors, __global float* gate_A, const int gate_ASize,
__global float* grad_gate_A, __global float* memory_state, const int memory_stateSize, __global
float* grad_memory_state, __global float* output, const int outputSize, __global float*
grad_output, const float gate_A, __global const float* saved_memory_state) {

^
<kernel>:175:165: note: previous declaration is here
        __kernel void text_forward_7829587341497687853(__global float* gate_vectors, const int
gate_vectorsSize, __global float* grad_gate_vectors, __global float* gate_A, const int gate_ASize,
__global float* grad_gate_A, __global float* memory_state, const int memory_stateSize, __global
float* grad_memory_state, __global float* output, const int outputSize, __global float*
grad_output, const float gate_A, __global const float* saved_memory_state) {

^
<kernel>:175:404: error: parameter name omitted
        __kernel void text_forward_7829587341497687853(__global float* gate_vectors, const int
gate_vectorsSize, __global float* grad_gate_vectors, __global float* gate_A, const int gate_ASize,
__global float* grad_gate_A, __global float* memory_state, const int memory_stateSize, __global
float* grad_memory_state, __global float* output, const int outputSize, __global float*
grad_output, const float gate_A, __global const float* saved_memory_state) {

^
<kernel>:178:73: error: invalid operands to binary expression ('__global float' and '__global float
*')
    if (tid < memory_stateSize) { memory_state[tid] = memory_state[tid] * gate_A; }
                                  ~~~~~~~~~~~~~~~~~~ ^ ~~~~~~~
<kernel>:179:61: error: invalid operands to binary expression ('__global float' and '__global float
*')
    if (tid < outputSize) { output[tid] = memory_state[tid] + gate_A; }
                            ~~~~~~~~~~~~~~~~~~ ^ ~~~~~~~

(options: -I C:\Users\Дмитрий\AppData\Local\Programs\Python\Python313\Lib\site-packages\
pyopencl\cl)
PS C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests>
```

Давай реализуем метод __call2__ (self,program_text:str)
который будет исправлять проблемы первой версии.
Исходим из того что каждая инструкция идет с новой строки → разделяем входную строку по символу переноса «\n»
Ко всем упоминанием тензоров добавляем „[tid]“ (“X=X+Y” → “X[tid]=X[tid]+Y[tid]”)
Заменяем все «nncllloc.» на «nn_»
Все параметры заменяем их значениями (поскольку в момент вызовы значения уже есть) (к примеру q =0.3 а в инструкции идет «R/=q» → «R[tid]/=0.3»
В полученных инструкциях проверяем:

1. Наличие хотябы 1 тензора (иначе получаеться что считаем в пустую или имеем комментарий)

2.Проверяем наличие знака равно. Если его нет , то должно быть 2 одинаковых мат знака (++ или --) . Если оно есть переходим к шагу 3.

3. Если знак = есть разделяем строку по нему. Если перед ним нет знаков «+-*.%» то переходим к шагу 4.

4.Проверяем осьавшуюся часть на наличие арефметических знаков «+-*/%» и то что миежду ними идут толко числа (не попали строковые значения).
Если в процессе проверки строки обнаружилась ошибка, кидаем исключение).
Если строка валидна, обрачиваем ее в конструкцию ваида:
f”if(tid<{названиеТензораКоторому присваиваем результат}>Size)”+<проверяемая строка>

Когда мы прошдлись по всем инструкциям, дописываем в начало имя ядра и аргументы ядра в формате:
<имя тензора>,<имя тензора>Size,….){
int tid={self.warp_size}*get_global_id(0)+get_global_id(1);\n

Пока напиши только этот код , обратным проходом займемся после.